

Computer Architecture

(EE 371 / CE 321 / CS 330)

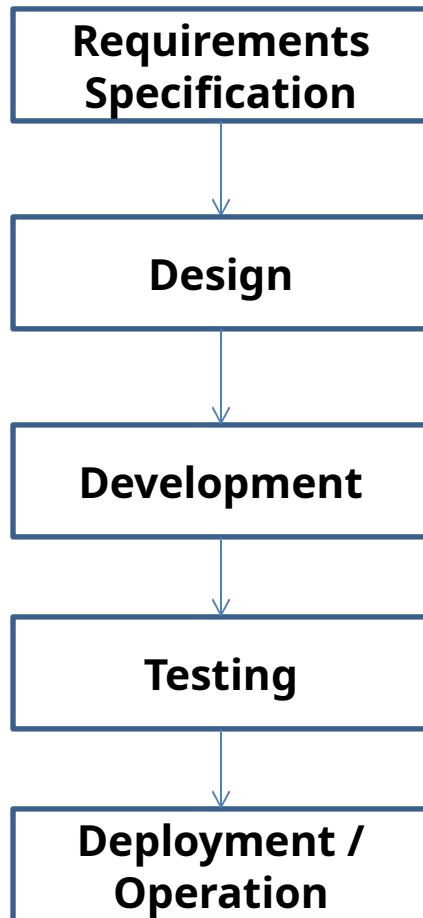
Dr. Farhan Khan
Assistant Professor,
Dhanani School of Science & Engineering,
Habib University

Outline: Unit 1

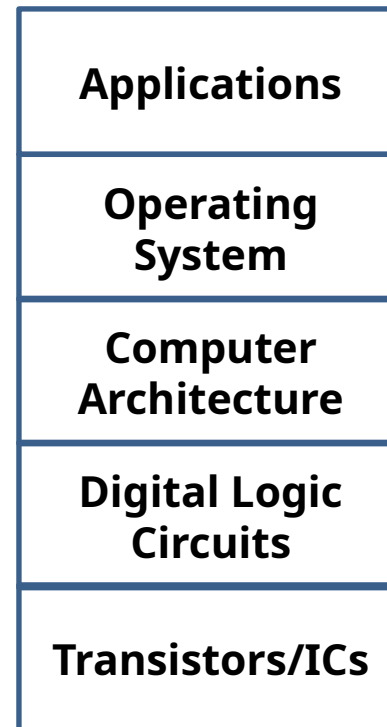
- Computer Architecture: The Big Picture
- Defining Performance
- Measuring Performance

Computer Engineering: The Big Picture

Process View

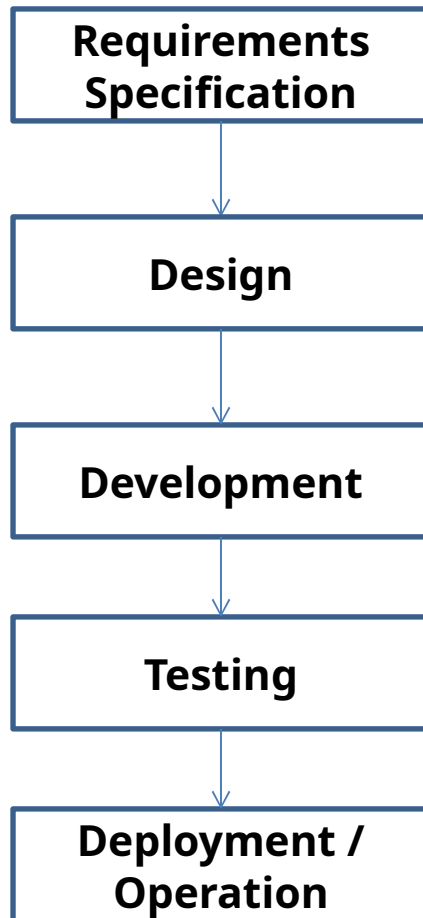


Structural View

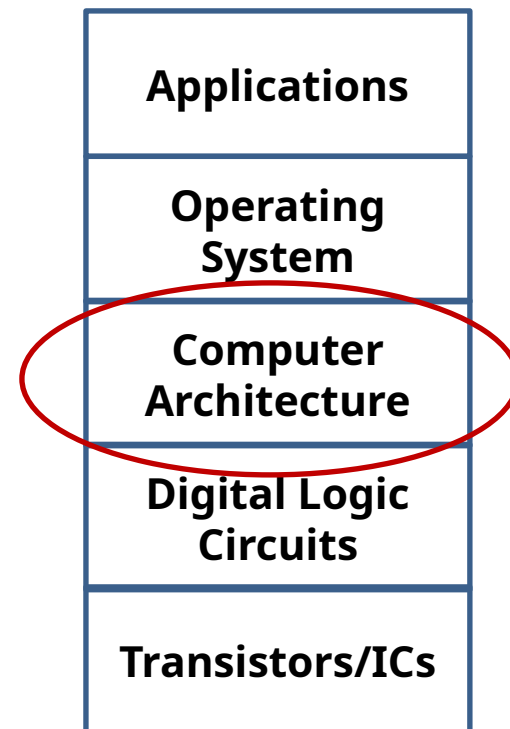


Computer Engineering: The Big Picture

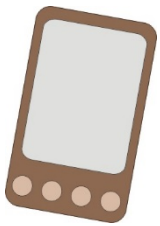
Process View



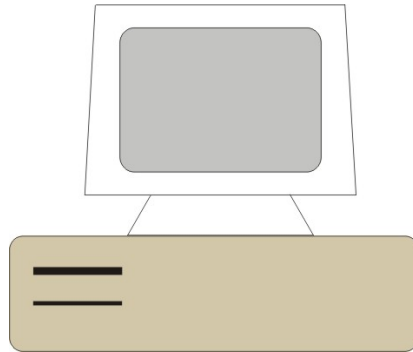
Structural View



"Computer": Examples



=



=



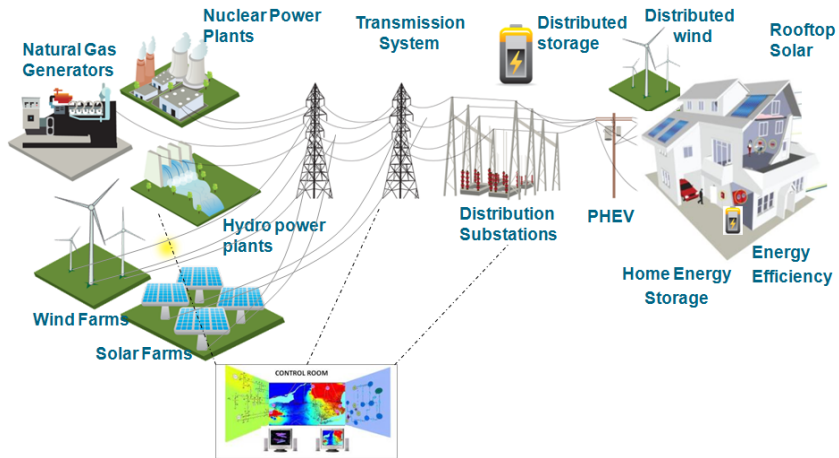
Cellphone

Workstation

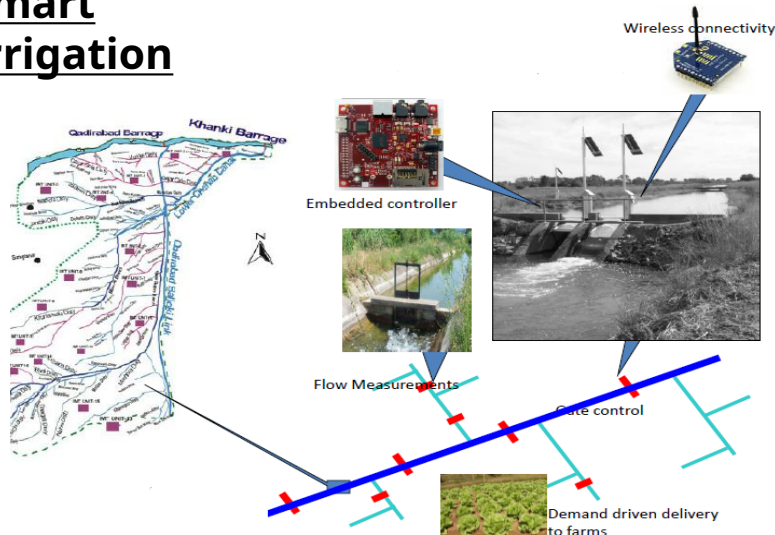
Supercomputer

"Computer": Examples

Smart Grid



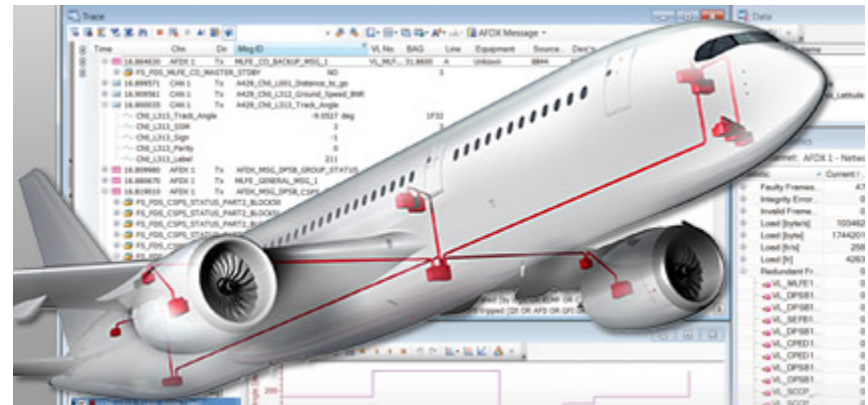
Smart Irrigation



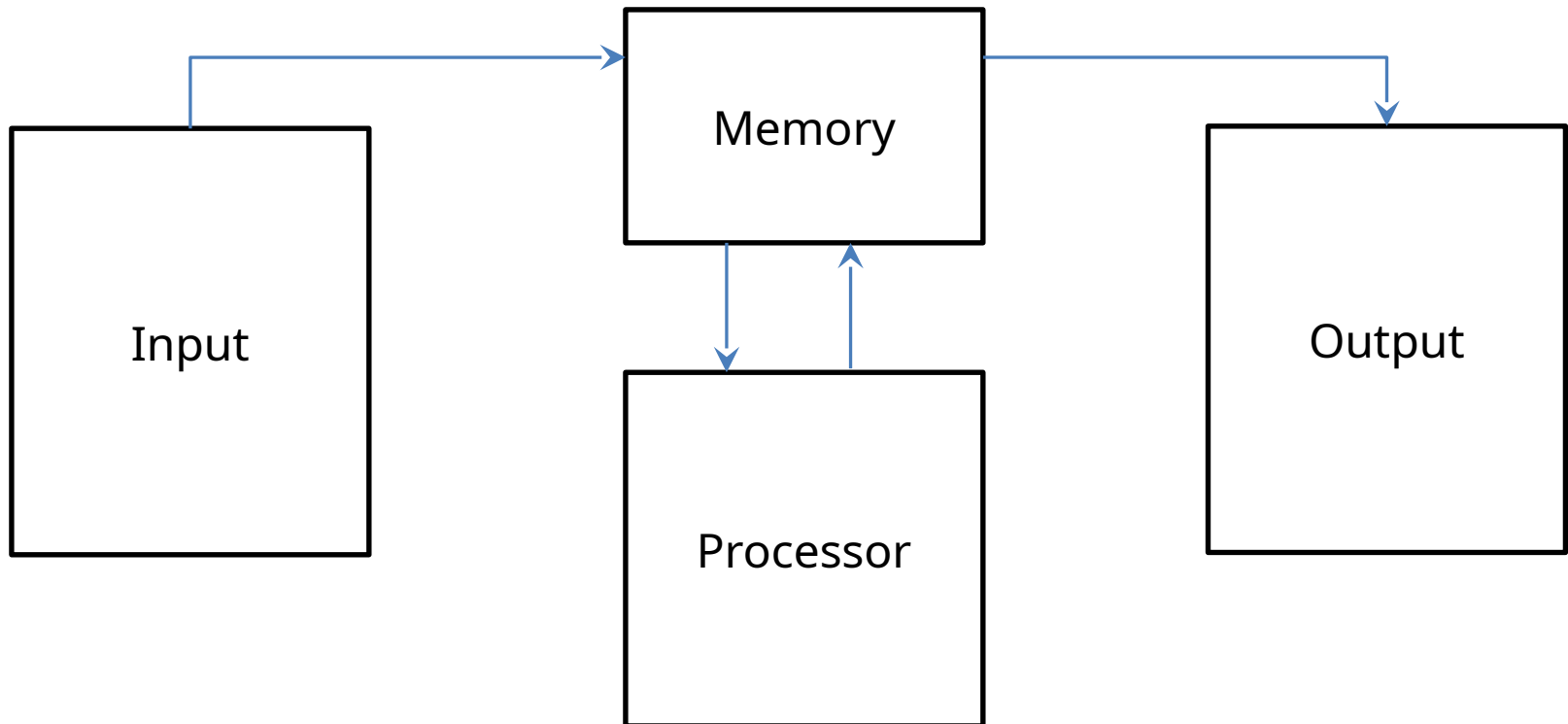
Automotive



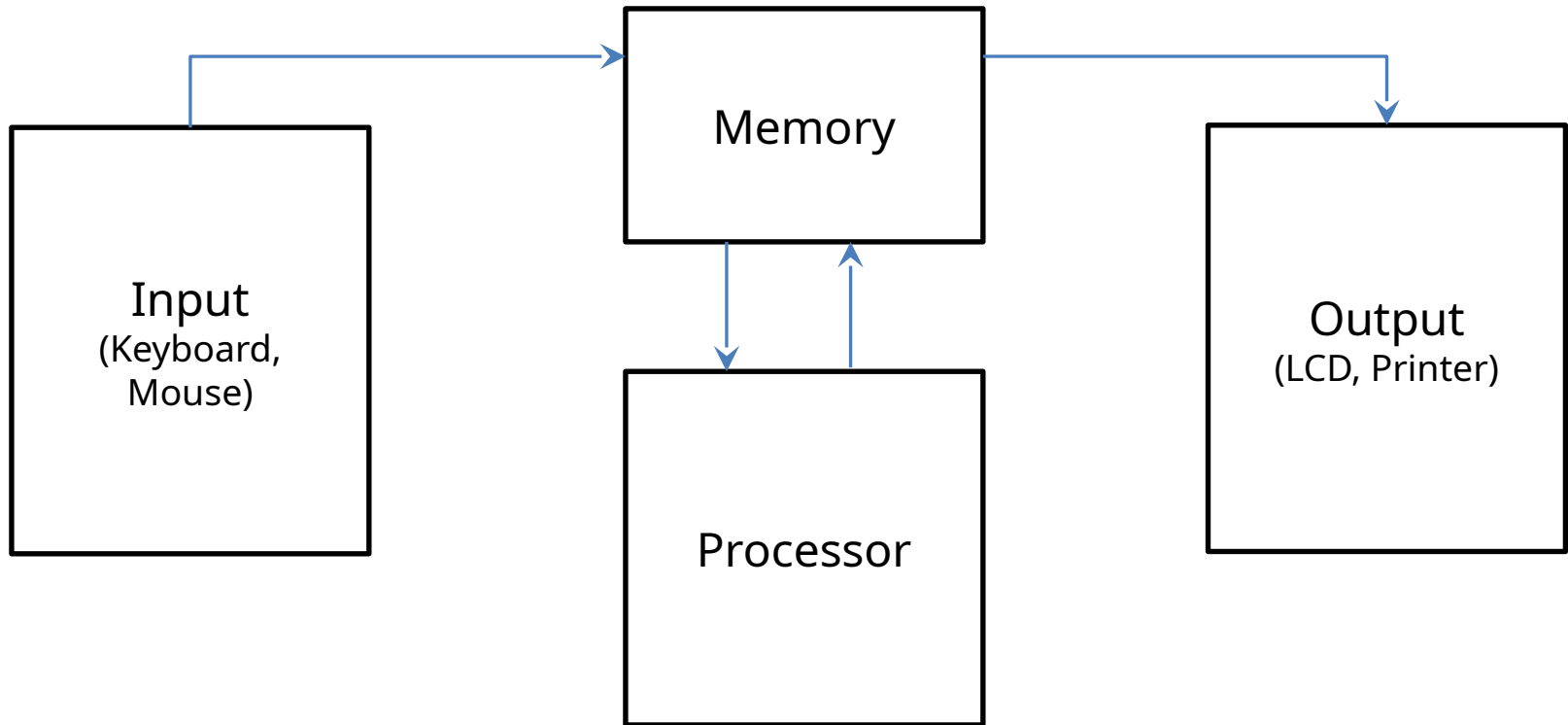
Avionics



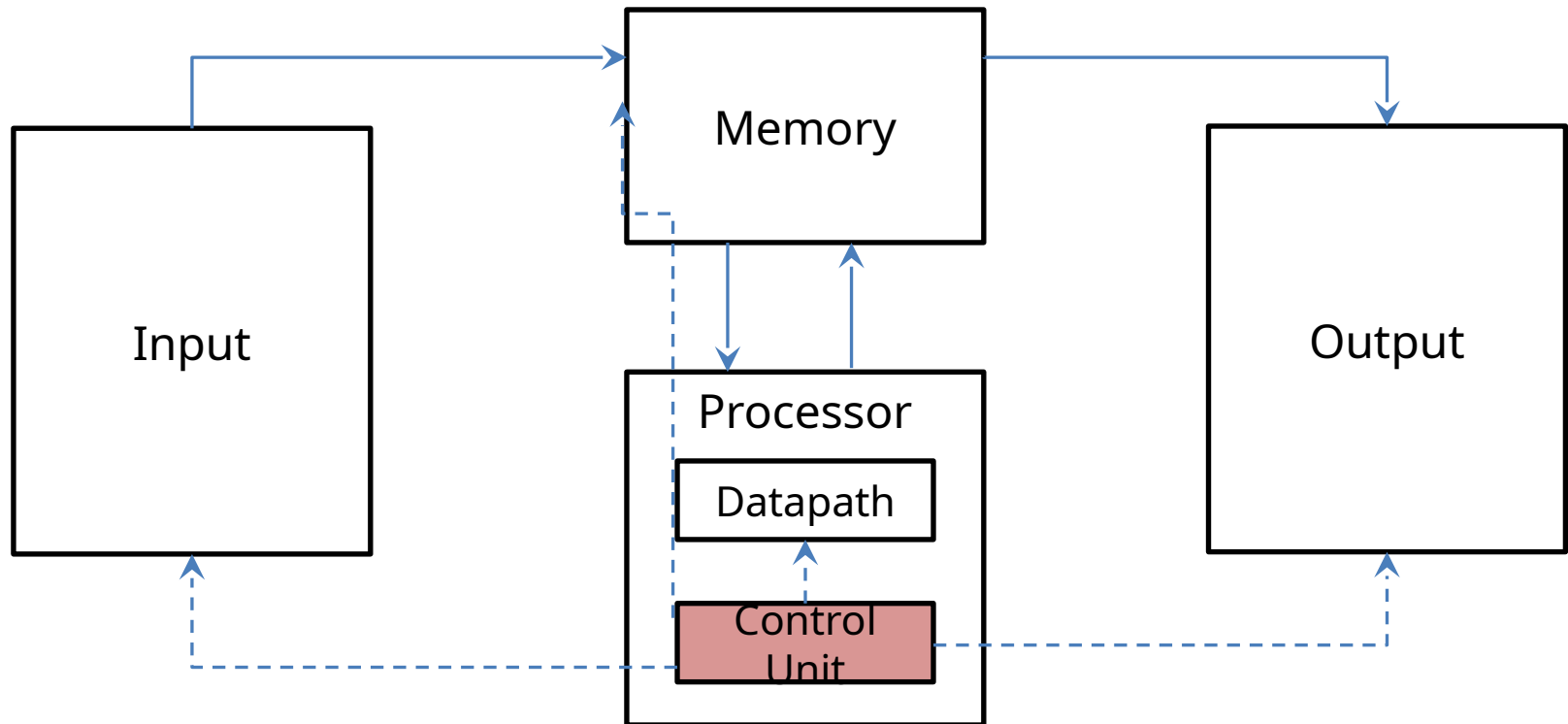
Computer Architecture: The Big Picture



Computer Architecture: The Big Picture



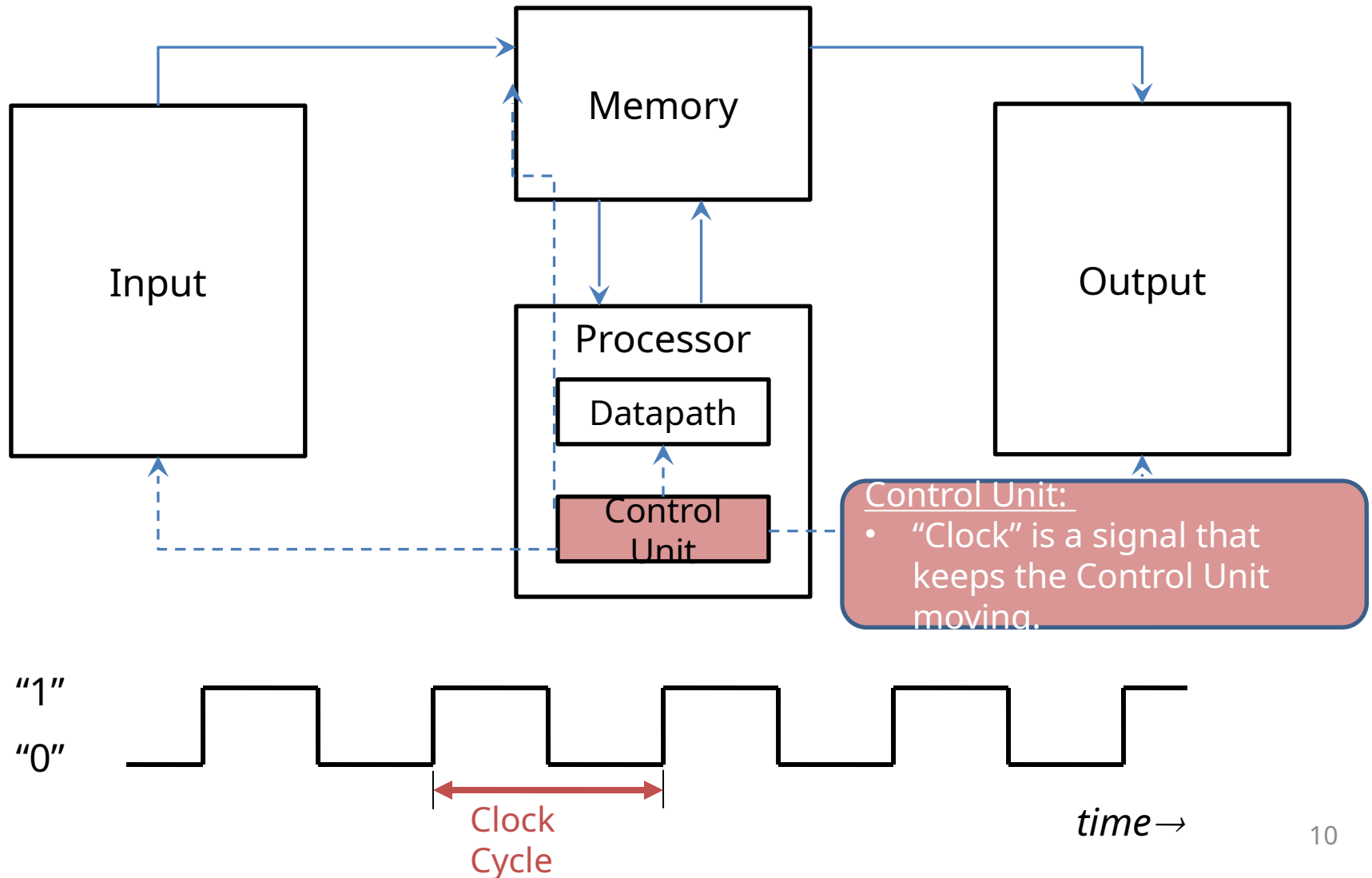
Computer Architecture: The Big Picture



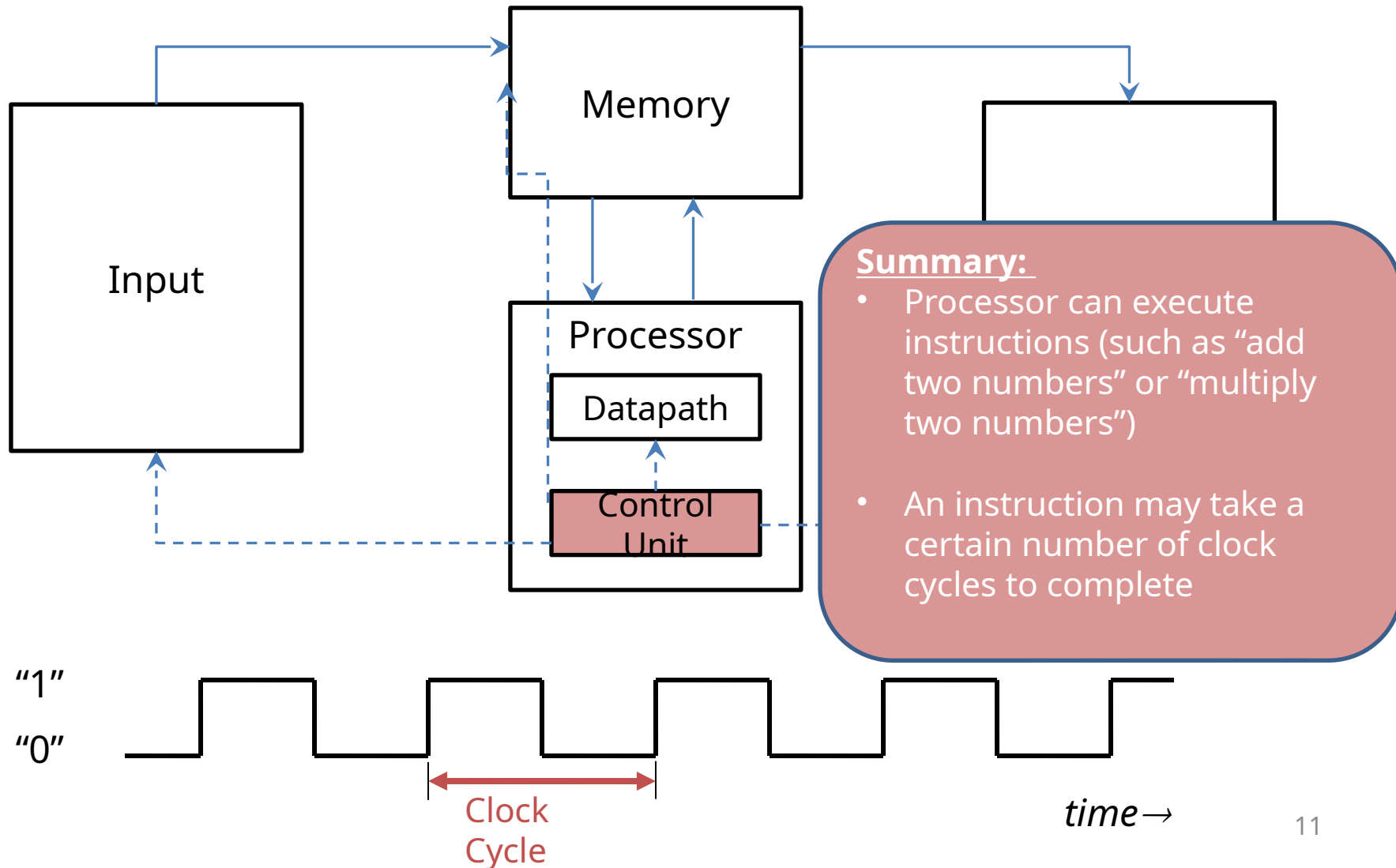
Control Unit:

- Reads an instruction from the Memory
- Interprets the instruction and generates signals that tell other components what to do

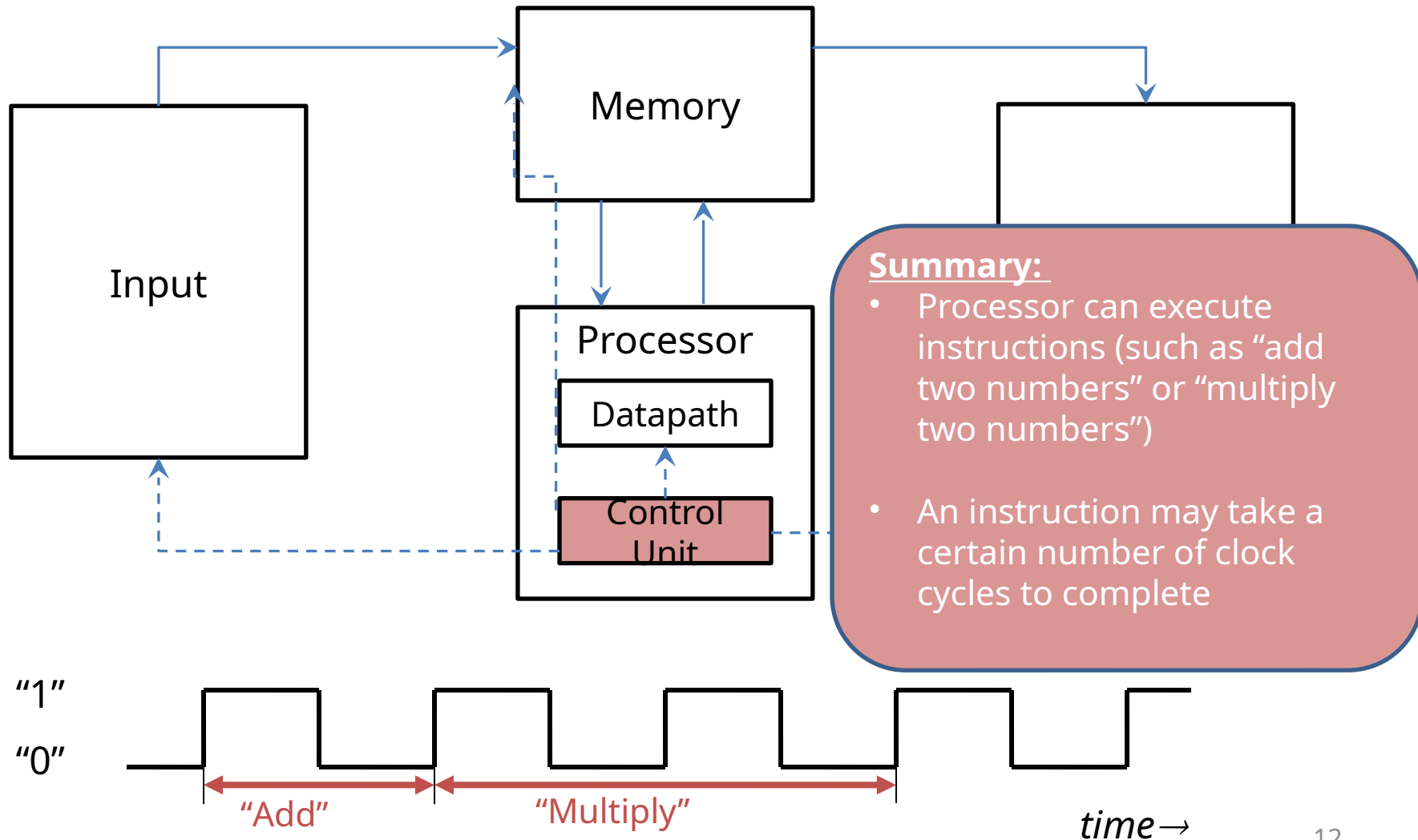
Computer Architecture: The Big Picture



Computer Architecture: The Big Picture



Computer Architecture: The Big Picture



Computer Architecture: The Big Picture

High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

Compiler

Assembly
language
program
(for RISC-V)

```
swap:
  slli x6, x11, 3
  add  x6, x10, x6
  ld   x5, 0(x6)
  ld   x7, 8(x6)
  sd   x7, 0(x6)
  sd   x5, 8(x6)
  jalr x0, 0(x1)
```

Assembler

Binary machine
language
program
(for RISC-V)

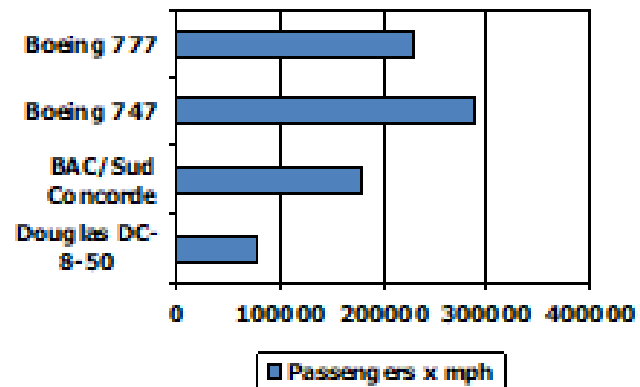
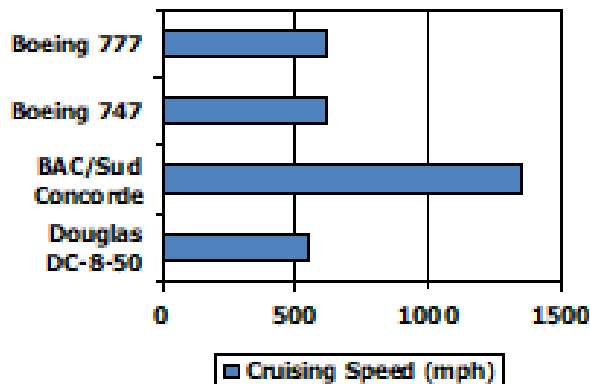
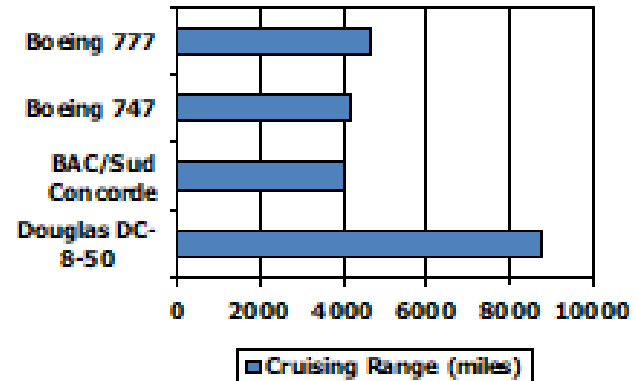
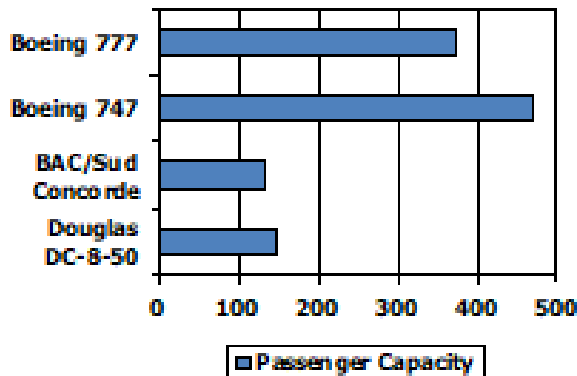
```
00000000001101011001001100010011
00000000011001010000001100110011
00000000000000110011001010000011
00000000100000110011001110000011
00000000011100110011000000100011
00000000010100110011010000100011
0000000000000000100000001100111
```

Defining “Performance”

*When we say one computer has better performance than another,
what do we mean?*

Defining "Performance"

- Which airplane has the best performance?



Defining “Performance”

Individual User

- Response Time / Execution Time
 - The time between start and completion of a task

Datacenter Manager

- Throughput
 - The total amount of work done in a given time
 - For example, tasks/transactions per hour

Defining “Performance”

Individual User

- Response Time / Execution Time
 - The time between start and completion of a task

Datacenter Manager

- Throughput
 - The total amount of work done in a give time
 - For example, tasks/transactions per hour

We will need different performance metrics for personal mobile devices, which are more focused on response time, versus servers, which are more focused on throughput.

Interplay between “Response Time” and “Throughput”

Do the following changes to a computer system increase throughput, decrease response time, or both?

1. Replacing the processor in a computer with a faster version
2. Adding additional processors to a system that uses multiple processors for separate tasks—for example, searching the web

Interplay between “Response Time” and Throughput

Do the following changes to a computer system increase throughput, decrease response time, or both?

1. Replacing the processor in a computer with a faster version
2. Adding additional processors to a system that uses multiple processors for separate tasks—for example, searching the web

Decreasing response time almost always improves throughput. Hence, in case 1, both response time and throughput are improved. In case 2, no one task gets work done faster, so only throughput increases.

If, however, the demand for processing in the second case was almost as large as the throughput, the system might force requests to queue up. In this case, increasing the throughput could also improve response time, since it would reduce the waiting time in the queue. Thus, in many real computer systems, changing either execution time or throughput often affects the other.

Defining “Performance”

Individual User

- Response Time / Execution Time
 - The time between start and completion of a task

Datacenter Manager

- Throughput
 - The total amount of work done in a give time
 - For example,

In this course, we will primarily be concerned with Response Time / Execution Time as a measure of Performance

r hour

We will need different performance metrics for personal mobile devices, which are more focused on response time, versus servers, which are more focused on throughput.

Defining “Performance”

- Performance of a Computer X is defined in terms of the execution time of a program on Computer X:

$$Performance_X = \frac{1}{Execution\ Time_X}$$

Defining “Performance”

- Performance of a Computer X is defined in terms of the execution time of a program on Computer X:

$$\text{Performance}_X = \frac{1}{\text{ExecutionTime}_X}$$

$$\begin{array}{l} \text{Performance}_X > \text{Performance}_Y \\ \frac{1}{\text{Execution time}_X} > \frac{1}{\text{Execution time}_Y} \\ \text{Execution time}_Y > \text{Execution time}_X \end{array}$$

Defining “Performance”

- Performance of a Computer X is defined in terms of the execution time of a program on Computer X:

$$\text{Performance}_X = \frac{1}{\text{ExecutionTime}_X}$$

- Computer X is n times faster than Computer Y:

$$\frac{\text{Performance}_X}{\text{Performance}_Y} = n$$

$$\frac{\text{Performance}_X}{\text{Performance}_Y} = \frac{\text{Execution time}_Y}{\text{Execution time}_X} = n$$

Measuring “Execution Time”

- **Elapsed time**

- Total response time, including all aspects
- Processing, I/O, OS overhead, idle time
- Determines system performance

- **CPU time**

- Time spent processing a given job
- Discounts I/O time, other jobs' shares
- Comprises user CPU time and system CPU time

Defining “Performance”

- Performance of a Computer X is defined in terms of the execution time of a program on Computer X:

$$\text{Performance}_X = \frac{1}{\text{Execution Time}_X}$$

$$\text{System Performance}_X = \frac{1}{\text{Execution Time in Elapsed Time}_X}$$

$$\text{CPU Performance}_X = \frac{1}{\text{Execution Time in CPU Time}_X}$$

Defining “Performance”

- Performance of a Computer X is defined in terms of the execution time of a program on Computer X:

In this course, we will mostly focus on CPU Performance

$$\text{System Performance}_X = \frac{1}{\text{ExecutionTime in Elapsed Time}_X}$$

$$\text{CPU Performance}_X = \frac{1}{\text{Execution Time in CPU Time}_X}$$

Classical Performance Equation

- Classical performance equation:

$$\text{CPU time} = \text{Instruction count} \times \text{CPI} \times \text{Clock cycle time}$$

where

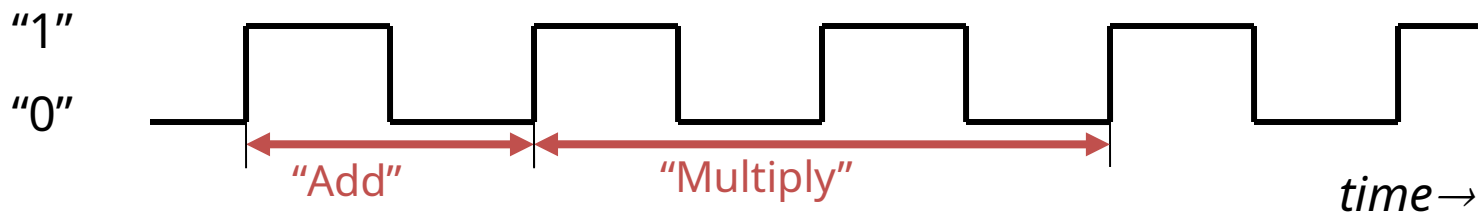
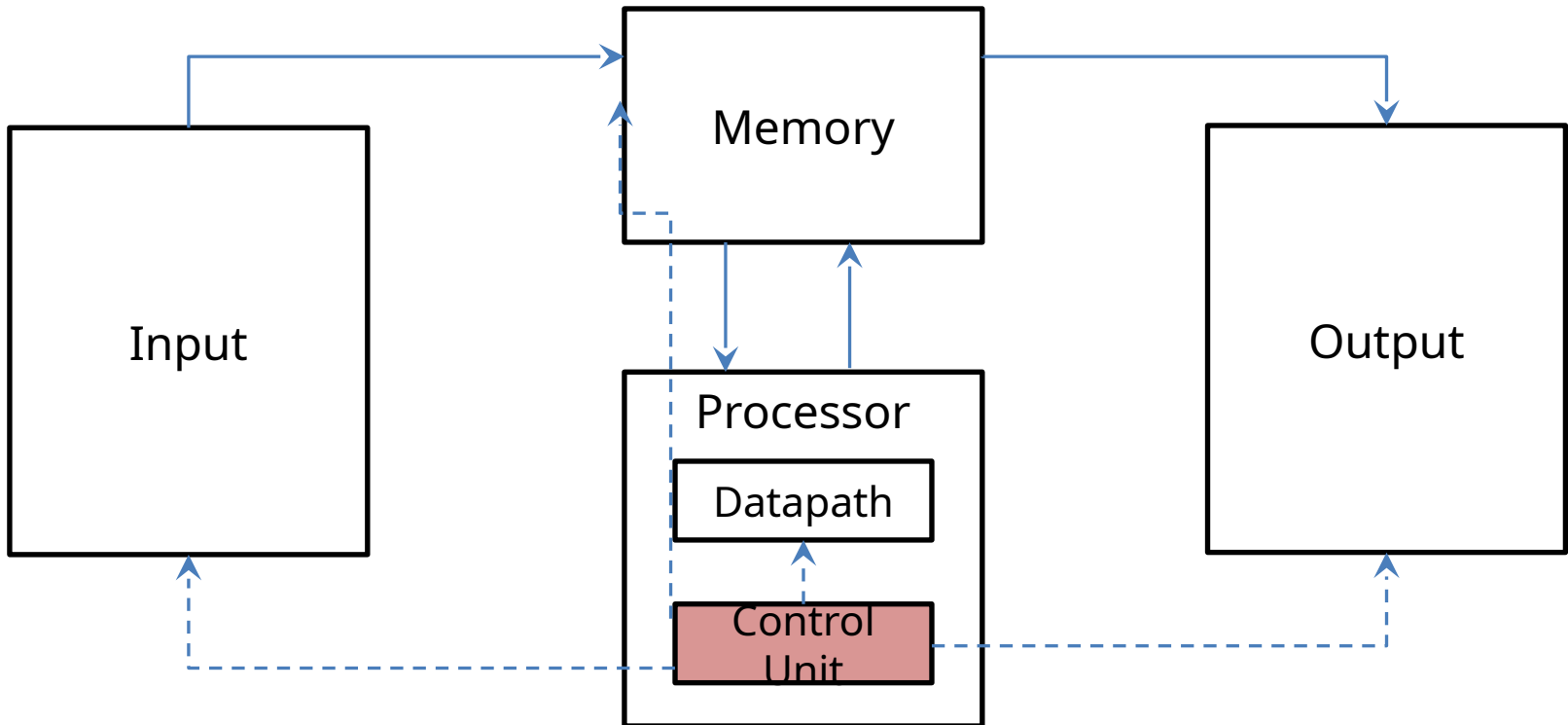
Instruction Count = Number of machine instruction in a program

CPI = Clock cycles per instruction

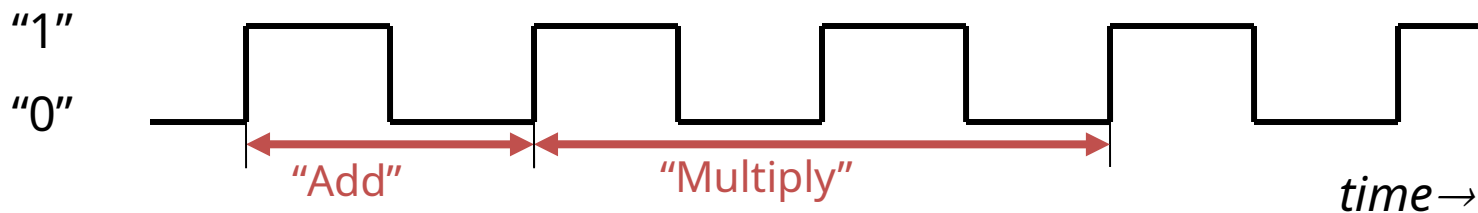
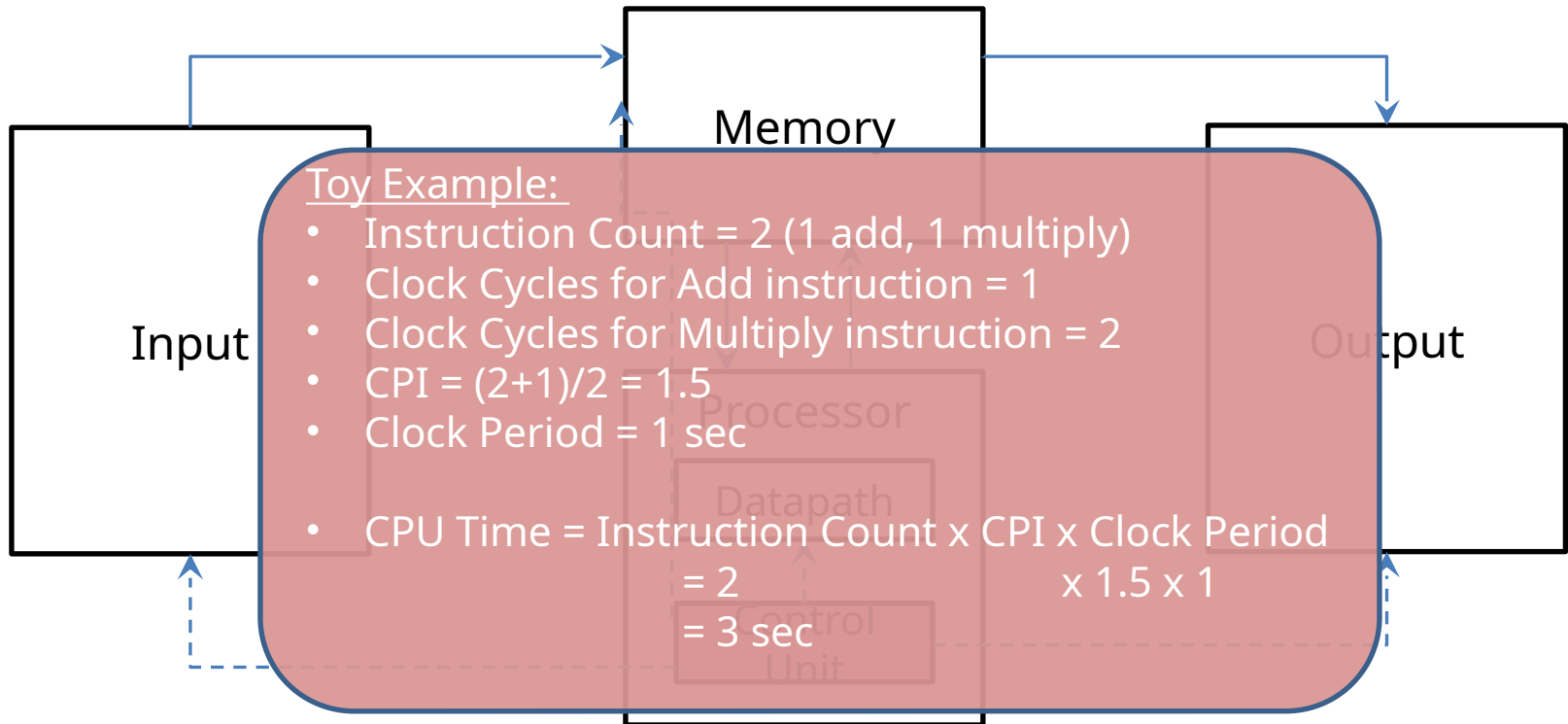
Clock cycle time = Clock Period = Time period of the clock signal

$$\text{CPU time} = \frac{\text{Instruction count} \times \text{CPI}}{\text{Clock rate}}$$

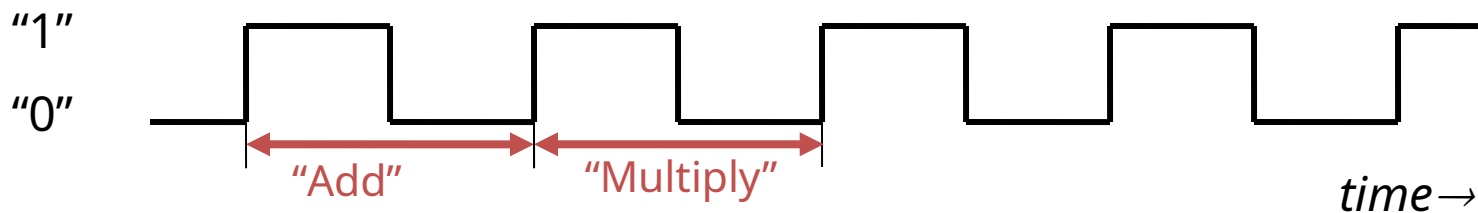
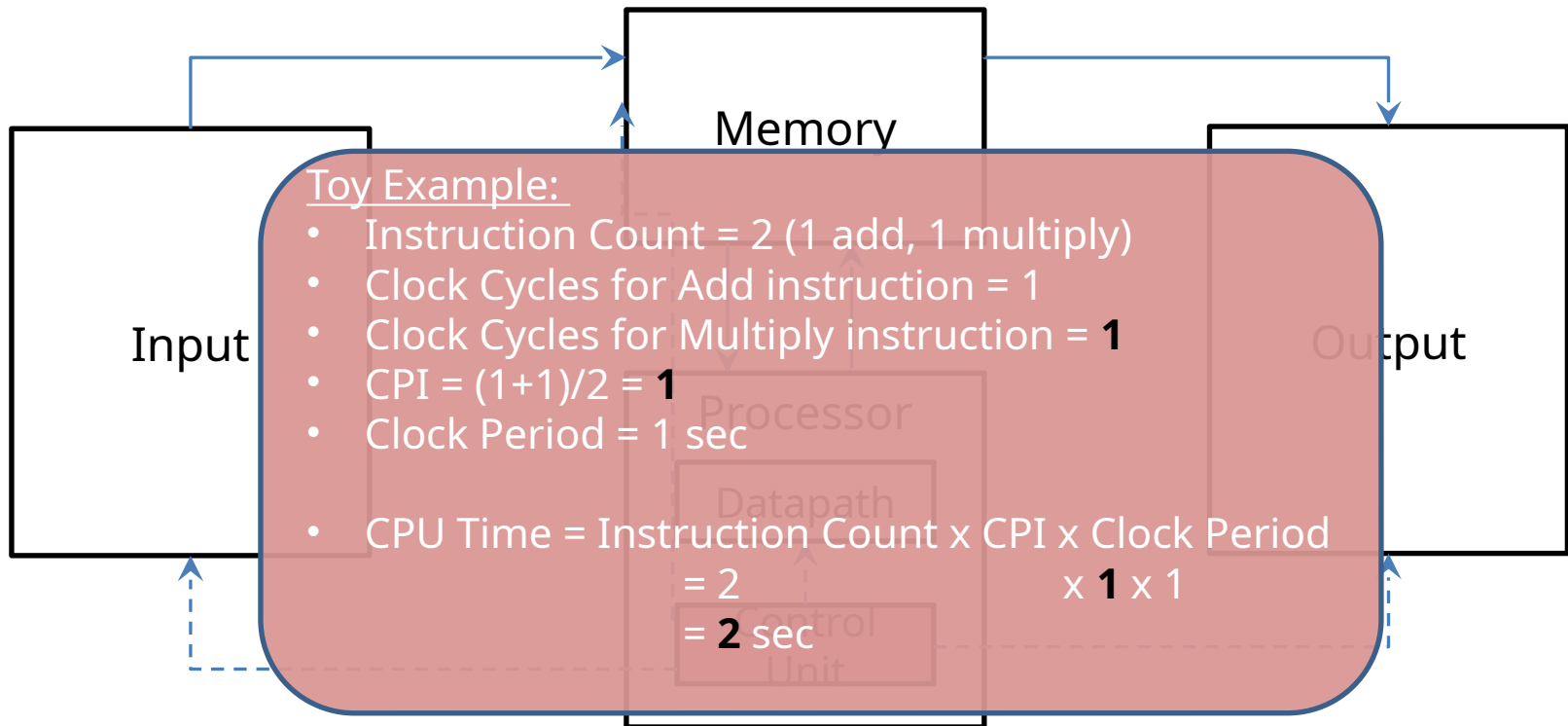
Classical Performance Equation



Classical Performance Equation



Improving Performance



Classical Performance Equation

- Classical performance equation:

$$\text{CPU time} = \text{Instruction count} \times \text{CPI} \times \text{Clock cycle time}$$

where

Instruction Count = Number of machine instruction in a program

CPI = Clock cycles per instruction

Clock cycle time = Clock Period = Time period of the clock signal

$$\text{CPU time} = \frac{\text{Instruction count} \times \text{CPI}}{\text{Clock rate}}$$

- These formulas are particularly useful because they separate the three key factors that affect performance.
- We can use these formulas to compare two different implementations or to evaluate a design alternative if we know its impact on these three parameters.

Classical Performance Equation

- Classical performance equation:

$$\text{CPU time} = \text{Instruction count} \times \text{CPI} \times \text{Clock cycle time}$$

where

Instruction Count = Number of machine instruction in a program

CPI = Clock cycles per instruction

Clock cycle time = Clock Period = Time period of the clock signal

$$\text{CPU time} = \frac{\text{Instruction count} \times \text{CPI}}{\text{Clock rate}}$$

- Performance depends on
 - Algorithm: affects IC, possibly CPI
 - Programming language: affects IC, CPI
 - Compiler: affects IC, CPI

Instruction Set

- Instruction Set
 - The vocabulary of instructions a computer can understand
- Different Computers have different instruction sets
 - But with many common aspects
- Examples
 - RISC-V
 - MIPS
 - Intel x86

Categories of RISC-V Instructions

- Arithmetic
- Data Transfer
- Logical
- Shift
- Conditional Branch
- Unconditional Branch

Arithmetic Instructions of RISC-V

Category	Instruction	Example	Meaning	Comments
Arithmetic	Add	<code>add x5, x6, x7</code>	$x5 = x6 + x7$	Three register operands; add
	Subtract	<code>sub x5, x6, x7</code>	$x5 = x6 - x7$	Three register operands; subtract
	Add immediate	<code>addi x5, x6, 20</code>	$x5 = x6 + 20$	Used to add constants

The Two Camps: CISC vs RISC

Complex Instruction Set Computer

- Includes multi-clock complex instructions
- Memory-to-memory: LOAD and STORE incorporated in instructions
- Small code sizes
- Transistors used for storing complex instructions
- Intel x86
- Example:
MULT M[5], M[3], M[2]

Reduced Instruction Set Computer

- Single-clock, reduced instruction only
- Register to register: "LOAD" and "STORE" are independent instructions
- Large code sizes
- Spends more transistors on memory registers
- RISC-V, MIPS, ARM, PowerPC
- Example:
LOAD R1, M[3]
LOAD R2, M[2]
MUL R3, R1, R2
STORE M[5], R3

CISC vs RISC

- Classical performance equation:

$$\text{CPU time} = \text{Instruction count} \times \text{CPI} \times \text{Clock cycle time}$$

where

Instruction Count = Number of machine instruction in a program

CPI = Clock cycles per instruction

Clock cycle time = Clock Period = Time period of the clock signal

- The CISC approach attempts to **minimize the number of instructions per program**, sacrificing the number of cycles per instruction.
- RISC does the opposite, **reducing the cycles per instruction** at the cost of the number of instructions per program.

Example

Suppose we have two implementations of the same instruction set architecture. Computer A has a clock cycle time of 250 ps and a CPI of 2.0 for some program, and computer B has a clock cycle time of 500 ps and a CPI of 1.2 for the same program. Which computer is faster for this program and by how much?

Example

Suppose we have two implementations of the same instruction set architecture. Computer A has a clock cycle time of 250 ps and a CPI of 2.0 for some program, and computer B has a clock cycle time of 500 ps and a CPI of 1.2 for the same program. Which computer is faster for this program and by how much?

We know that each computer executes the same number of instructions for the program; let's call this number I . First, find the number of processor clock cycles for each computer:

$$\text{CPU clock cycles}_A = I \times 2.0$$

$$\text{CPU clock cycles}_B = I \times 1.2$$

Now we can compute the CPU time for each computer:

$$\begin{aligned}\text{CPU time}_A &= \text{CPU clock cycles}_A \times \text{Clock cycle time} \\ &= I \times 2.0 \times 250 \text{ ps} = 500 \times I \text{ ps}\end{aligned}$$

Likewise, for B:

$$\text{CPU time}_B = I \times 1.2 \times 500 \text{ ps} = 600 \times I \text{ ps}$$

Clearly, computer A is faster. The amount faster is given by the ratio of the execution times:

$$\frac{\text{CPU performance}_A}{\text{CPU performance}_B} = \frac{\text{Execution time}_B}{\text{Execution time}_A} = \frac{600 \times I \text{ ps}}{500 \times I \text{ ps}} = 1.2$$

We can conclude that computer A is 1.2 times as fast as computer B for this program.

Example

A compiler designer is trying to decide between two code sequences for a computer. The hardware designers have supplied the following facts:

	CPI for each instruction class		
	A	B	C
CPI	1	2	3

For a particular high-level language statement, the compiler writer is considering two code sequences that require the following instruction counts:

Code sequence	Instruction counts for each instruction class		
	A	B	C
1	2	1	2
2	4	1	1

Which code sequence executes the most instructions? Which will be faster? What is the CPI for each sequence?

Example

A compiler designer is trying to decide between two code sequences for a computer. The hardware designers have supplied the following facts:

	CPI for each instruction class		
	A	B	C
CPI	1	2	3

For a particular high-level language statement, the compiler writer is considering two code sequences that require the following instruction counts:

Code sequence	Instruction counts for each instruction class		
	A	B	C
1	2	1	2
2	4	1	1

Which code sequence executes the most instructions? Which will be faster? What is the CPI for each sequence?

$$\text{CPU clock cycles}_1 = (2 \times 1) + (1 \times 2) + (2 \times 3) = 2 + 2 + 6 = 10 \text{ cycles}$$

$$\text{CPU clock cycles}_2 = (4 \times 1) + (1 \times 2) + (1 \times 3) = 4 + 2 + 3 = 9 \text{ cycles}$$

So code sequence 2 is faster, even though it executes one extra instruction. Since code sequence 2 takes fewer overall clock cycles but has more instructions, it must have a lower CPI. The CPI values can be computed by

$$\text{CPI} = \frac{\text{CPU clock cycles}}{\text{Instruction count}}$$

$$\text{CPI}_1 = \frac{\text{CPU clock cycles}_1}{\text{Instruction count}_1} = \frac{10}{5} = 2.0$$

$$\text{CPI}_2 = \frac{\text{CPU clock cycles}_2}{\text{Instruction count}_2} = \frac{9}{6} = 1.5$$

Example

A given application written in Java runs 15 seconds on a desktop processor. A new Java compiler is released that requires only 0.6 as many instructions as the old compiler. Unfortunately, it increases the CPI by 1.1. How fast can we expect the application to run using this new compiler? Pick the right answer from the three choices below:

a. $\frac{15 \times 0.6}{1.1} = 8.2 \text{ sec}$

b. $15 \times 0.6 \times 1.1 = 9.9 \text{ sec}$

c. $\frac{1.5 \times 1.1}{0.6} = 27.5 \text{ sec}$